

BLM4522 – Ağ Tabanlı Paralel Dağıtım Sistemleri

Hasan Kalın - 20291267

GitHub: @hkl4

Drive Video: <https://drive.google.com/drive/folders/13OLp1ahdT-W1hLpW4e53MtYazaeDaf?usp=sharing>

PROJE 5: VERİTABANI YÜKSELTMESİ VE SÜRÜM YÖNETİMİ

1. Giriş

Bu proje kapsamında PostgreSQL ve DBeaver kullanılarak e-ticaret veritabanı (performans_proje) üzerinde şema yükseltme ve sürüm yönetimi uygulandı. Mevcut v1 şeması belgelendi, DDL Event Trigger ile tüm şema değişiklikleri otomatik olarak loglandı, v1'den v2'ye migration gerçekleştirildi, geri dönüş (rollback) planı test edildi ve migration öncesi/sonrası veri bütünlüğü doğrulandı.

2. Şema Analizi (v1 Durumu)

```
-- 1. Mevcut tabloları listele
SELECT table_name, table_type
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;

-- 2. Her tablonun sütun yapısını görüntüle
SELECT table_name, column_name, data_type, character_maximum_length
FROM information_schema.columns
WHERE table_schema = 'public'
ORDER BY table_name, ordinal_position;

-- 3. Mevcut indexleri listele
SELECT indexname, tablename, indexdef
FROM pg_indexes
WHERE schemaname = 'public'
ORDER BY tablename;

-- 4. Tablo boyutları
SELECT
    relname AS tablo_adi,
    pg_size_pretty(pg_total_relation_size(relid)) AS toplam_boyut,
    n_live_tup AS kayıt_sayisi
FROM pg_stat_user_tables
ORDER BY pg_total_relation_size(relid) DESC;
```

Migration öncesinde veritabanının v1 şeması analiz edildi. Mevcut tablolar, sütun yapıları, indexler ve tablo boyutları sorgulandı. Bu adım migration için referans noktası oluşturdu.

Tablo	Toplam Boyut	Kayıt Sayısı
orders	210 MB	1.000.000
customers	144 kB	1.000
products	88 kB	500

Mevcut Index	Tablo
idx_orders_order_date	orders
idx_orders_customer_id	orders
idx_orders_product_id	orders

3. DDL Trigger Kurulumu

```
-- 1. Şema değişikliklerini loglayacak tablo
CREATE TABLE schema_log (
    id SERIAL PRIMARY KEY,
    islem_zamani TIMESTAMP DEFAULT NOW(),
    kullanıcı TEXT DEFAULT CURRENT_USER,
    islem_turu TEXT,
    nesne_turu TEXT,
    nesne_adi TEXT,
    sorgu TEXT
);

-- 2. DDL trigger fonksiyonu
CREATE OR REPLACE FUNCTION schema_degisiklik_log()
RETURNS EVENT_TRIGGER AS $$
DECLARE
    obj RECORD;
BEGIN
    FOR obj IN SELECT * FROM pg_event_trigger_ddl_commands()
    LOOP
        INSERT INTO schema_log (islem_turu, nesne_turu, nesne_adi, sorgu)
        VALUES (
            TG_EVENT,
            obj.object_type,
            obj.object_identity,
            current_query()
        );
    END LOOP;
END;
$$ LANGUAGE plpgsql;

-- 3. Event trigger'ı oluştur
CREATE EVENT TRIGGER schema_degisiklik_trigger
ON ddl_command_end
EXECUTE FUNCTION schema_degisiklik_log();

-- 4. Trigger'ı doğrula
SELECT evtname, evttype, evtenabled
FROM pg_event_trigger
WHERE evtname = 'schema_degisiklik_trigger';
```

Tüm şema değişikliklerini otomatik olarak kayıt altına almak için schema_log tablosu ve DDL Event Trigger oluşturuldu. Trigger, ddl_command_end olayında tetiklenerek her ALTER, CREATE ve DROP işlemini işlemi yapan kullanıcı, zaman damgası ve çalıştırılan SQL ile birlikte loglar.

Alan	Açıklama
islem_zamani	İşlemin gerçekleştiği zaman damgası
kullanici	İşlemi yapan PostgreSQL kullanıcısı
islem_turu	DDL işlem türü (ddl_command_end)
nesne_turu	table / index / sequence
nesne_adi	Değiştirilen nesnenin tam adı
sorgu	Çalıştırılan SQL sorgusu

Trigger doğrulama: schema_degisiklik_trigger aktif durumda (evtenabled: O)

4. Şema Versiyonlama Tablosu

```
-- Migration geçmişini tutacak tablo
CREATE TABLE schema_versiyon (
  id SERIAL PRIMARY KEY,
  versiyon VARCHAR(20),
  aciklama TEXT,
  uygulayan TEXT DEFAULT CURRENT_USER,
  uygulama_zamani TIMESTAMP DEFAULT NOW(),
  durum VARCHAR(20) DEFAULT 'BAŞARILI'
);

-- v1 başlangıç durumunu kaydet
INSERT INTO schema_versiyon (versiyon, aciklama)
VALUES ('v1.0.0', 'Başlangıç şeması: customers, orders, products');

SELECT * FROM schema_versiyon;
```

Migration geçmişini takip etmek için schema_versiyon tablosu oluşturuldu. Her migration uygulandığında bu tabloya versiyon numarası, açıklama, uygulayan kullanıcı, zaman damgası ve durum bilgisi kaydedilir. v1.0.0 başlangıç durumu ilk kayıt olarak eklendi.

Versiyon	Açıklama
v1.0.0	Başlangıç şeması: customers, orders, products

5. v1 → v2 Migration

```
-- 1. Migration öncesi test: mevcut kayıt sayılarını kaydet
CREATE TABLE migration_test_v1 AS
SELECT
  'orders' AS tablo, COUNT(*) AS kayıt_sayisi FROM orders
UNION ALL
SELECT 'customers', COUNT(*) FROM customers
UNION ALL
SELECT 'products', COUNT(*) FROM products;

-- 2. customers tablosuna yeni sütunlar ekle
ALTER TABLE customers
ADD COLUMN phone VARCHAR(20),
ADD COLUMN updated_at TIMESTAMP DEFAULT NOW(),
ADD COLUMN segment VARCHAR(20) DEFAULT 'standard';

-- 3. orders tablosuna yeni sütunlar ekle
ALTER TABLE orders
ADD COLUMN discount NUMERIC(5,2) DEFAULT 0,
ADD COLUMN shipping_cost NUMERIC(10,2) DEFAULT 0,
ADD COLUMN updated_at TIMESTAMP DEFAULT NOW();

-- 4. products tablosuna yeni sütunlar ekle
ALTER TABLE products
ADD COLUMN stock_quantity INTEGER DEFAULT 0,
ADD COLUMN updated_at TIMESTAMP DEFAULT NOW();

-- 5. Yeni index ekle
CREATE INDEX idx_orders_status ON orders(status);
CREATE INDEX idx_customers_country ON customers(country);
CREATE INDEX idx_products_category ON products(category);

-- 6. customers segment sütununu güncelle
UPDATE customers
SET segment = CASE
  WHEN country IN ('USA', 'Germany') THEN 'premium'
  WHEN country IN ('France', 'UK') THEN 'standard'
  ELSE 'basic'
END;

-- 7. Versiyon tablosunu güncelle
INSERT INTO schema_versiyon (versiyon, aciklama)
VALUES ('v2.0.0', 'customers/orders/products tablolarına yeni sütunlar eklendi, 3 yeni index eklendi');

-- 8. Migration sonrası doğrulama
SELECT * FROM schema_versiyon ORDER BY uygulama_zamani;
```

Migration başlamadan önce mevcut kayıt sayıları migration_test_v1 referans tablosuna kaydedildi. Ardından 3 tabloya toplam 8 yeni sütun eklendi, 3 yeni index oluşturuldu ve customers tablosundaki segment sütunu müşterinin bulunduğu ülkeye göre (premium, standard, basic) güncellendi.

Tablo	Eklenen Sütunlar
customers	phone, updated_at, segment
orders	discount, shipping_cost, updated_at
products	stock_quantity, updated_at

Yeni Index	Tablo
idx_orders_status	orders
idx_customers_country	customers
idx_products_category	products

6. DDL Log Doğrulama

```
-- 1. Tüm şema değişikliklerini görüntüle
SELECT * FROM schema_log ORDER BY islem_zamani;

-- 2. İşlem türüne göre özet
SELECT islem_turu, nesne_turu, COUNT(*) AS adet
FROM schema_log
GROUP BY islem_turu, nesne_turu
ORDER BY adet DESC;

-- 3. v2 sonrası yeni sütunları doğrula
SELECT table_name, column_name, data_type
FROM information_schema.columns
WHERE table_schema = 'public'
  AND table_name IN ('customers', 'orders', 'products')
  AND column_name IN ('phone', 'updated_at', 'segment', 'discount',
    'shipping_cost', 'stock_quantity')
ORDER BY table_name, column_name;

-- 4. Yeni indexleri doğrula
SELECT indexname, tablename
FROM pg_indexes
WHERE schemaname = 'public'
  AND indexname IN ('idx_orders_status', 'idx_customers_country', 'idx_products_category');
```

Migration sonrası schema_log tablosu sorgulandı. DDL trigger'ın tüm değişiklikleri başarıyla logladığı doğrulandı. schema_versiyon tablosunun oluşturulmasından v2 migration'ına kadar toplam 11 DDL işlemi kayıt altına alındı.

Nesne Türü	Adet	Açıklama
table	5	Oluşturulan/değiştirilen tablolar
index	4	Oluşturulan indexler
sequence	2	Otomatik oluşturulan sequence'lar

v2 ile eklenen 8 yeni sütun ve 3 yeni index information_schema üzerinden doğrulandı.

7. Geri Dönüş (Rollback) Planı

```
● -- 1. Migration öncesi kayıt sayılarını kontrol et
SELECT * FROM migration_test_v1;

● -- 2. v2'de eklenen indexleri kaldır
DROP INDEX idx_orders_status;
DROP INDEX idx_customers_country;
DROP INDEX idx_products_category;

● -- 3. v2'de eklenen sütunları kaldır
ALTER TABLE customers
  DROP COLUMN phone,
  DROP COLUMN updated_at,
  DROP COLUMN segment;

● ALTER TABLE orders
  DROP COLUMN discount,
  DROP COLUMN shipping_cost,
  DROP COLUMN updated_at;

● ALTER TABLE products
  DROP COLUMN stock_quantity,
  DROP COLUMN updated_at;

● -- 4. Rollback'i versiyon tablosuna kaydet
INSERT INTO schema_versiyon (versiyon, aciklama, durum)
VALUES ('v1.0.0', 'v2.0.0 geri alındı - eklenen sütunlar ve indexler kaldırıldı', 'tamamlandı');

● -- 5. Rollback sonrası sütunları doğrula
SELECT table_name, column_name, data_type
FROM information_schema.columns
WHERE table_schema = 'public'
AND table_name IN ('customers', 'orders', 'products')
ORDER BY table_name, ordinal_position;

● -- 6. Versiyon geçmişini görüntüle
SELECT * FROM schema_versiyon ORDER BY uygulama_zamani;
```

Migration sonrası geri dönüş planı test edildi. v2.0.0'da eklenen 3 index DROP INDEX ile, 8 sütun ise ALTER TABLE ... DROP COLUMN ile kaldırıldı. Tablolar orijinal v1.0.0 yapısına döndü ve kayıt sayıları korundu. Rollback işlemi schema_versiyon tablosuna ROLLBACK statüsüyle kaydedildi.

Versiyon	Açıklama
v1.0.0	Başlangıç şeması
v2.0.0	Yeni sütunlar ve indexler eklendi
v1.0.0	v2.0.0 geri alındı
v2.0.0	Test sonrası tekrar uygulandı

8. Migration Öncesi/Sonrası Test Senaryoları

```
● -- 1. v1 durumunu kaydet (referans nokta)
SELECT
  'v1' AS versiyon,
  COUNT(*) AS toplam_kayit,
  COUNT(DISTINCT customer_id) AS benzersiz_musteri,
  COUNT(DISTINCT product_id) AS benzersiz_urun,
  ROUND(AVG(total_price), 2) AS ort_siparis_tutari,
  MIN(order_date) AS ilk_siparis,
  MAX(order_date) AS son_siparis
FROM orders;

● -- 2. v2 migration'ı tekrar uygula
ALTER TABLE customers
  ADD COLUMN phone VARCHAR(20),
  ADD COLUMN updated_at TIMESTAMP DEFAULT NOW(),
  ADD COLUMN segment VARCHAR(20) DEFAULT 'standard';

● ALTER TABLE orders
  ADD COLUMN discount NUMERIC(5,2) DEFAULT 0,
  ADD COLUMN shipping_cost NUMERIC(10,2) DEFAULT 0,
  ADD COLUMN updated_at TIMESTAMP DEFAULT NOW();

● ALTER TABLE products
  ADD COLUMN stock_quantity INTEGER DEFAULT 0,
  ADD COLUMN updated_at TIMESTAMP DEFAULT NOW();

CREATE INDEX idx_orders_status ON orders(status);
CREATE INDEX idx_customers_country ON customers(country);
CREATE INDEX idx_products_category ON products(category);

● -- 3. v2 sonrası aynı sorguyu çalıştır
SELECT
  'v2' AS versiyon,
  COUNT(*) AS toplam_kayit,
  COUNT(DISTINCT customer_id) AS benzersiz_musteri,
  COUNT(DISTINCT product_id) AS benzersiz_urun,
  ROUND(AVG(total_price), 2) AS ort_siparis_tutari,
  MIN(order_date) AS ilk_siparis,
  MAX(order_date) AS son_siparis
FROM orders;

● -- 4. Veri bütünlüğü kontrolü: migration sonrası NULL kontrol
SELECT
  COUNT(*) AS toplam,
  COUNT(discount) AS dolu_discount,
  COUNT(shipping_cost) AS dolu_shipping,
  COUNT(updated_at) AS dolu_updated_at
FROM orders;

● -- 5. Versiyon tablosunu güncelle
INSERT INTO schema_versiyon (versiyon, aciklama)
VALUES ('v2.0.0', 'v2.0.0 test sonrası tekrar uygulandı - tüm testler başarılı');

● -- 6. Final versiyon geçmişi
SELECT * FROM schema_versiyon ORDER BY uygulama_zamani;
```

v1 ve v2 karşılaştırmalı test sorgusu çalıştırıldı. Migration'ın mevcut veriyi etkileyip etkilemediği incelendi. Ardından yeni sütunlarda NULL kontrol sorgusu çalıştırıldı.

Metrik	v1	
Toplam kayıt	1.000.000	1.000.000
Benzersiz müşteri	1.000	1.000
Benzersiz ürün	500	500
Ort. sipariş tutarı	2.512,25	2.512,25
İlk sipariş	2019-01-01	2019-01-01
Son sipariş	2023-12-06	2023-12-06

NULL kontrol: 1 milyon kaydın tamamında discount, shipping_cost ve updated_at sütunları DEFAULT değerleriyle dolu — migration veri kaybına veya NULL oluşumuna yol açmadı.

9. Sonuç

Bu proje kapsamında e-ticaret veritabanı üzerinde eksiksiz bir şema yükseltme ve sürüm yönetimi süreci uygulandı. v1 şeması analiz edilerek belgelendi. DDL Event Trigger ile tüm şema değişiklikleri otomatik olarak loglandı ve 11 DDL işlemi kayıt altına alındı. v1.0.0'dan v2.0.0'a migration ile 3 tabloya 8 yeni sütun ve 3 yeni index eklendi. Rollback planı test edilerek v1 şemasına başarıyla döndü. Migration öncesi/sonrası test sorguları ile veri bütünlüğünün korunduğu doğrulandı; tüm metrikler v1 ve v2'de birebir aynı çıktı.